DevOps Question Bank
Shell Scripting: 24 Real Interview Scripts

24 real interview questions - de-duplicated, topic-wise, Basic / Advanced / Scenario-Based
Every answer: plain-English idea · technical answer · example · what to say in the interview

By Priyanka Gupta — 25 September 2026

How to use this question bank

Each question has four parts:

In simple words (green box) - a plain-English analogy. Read this first; if you understand it, the technical part becomes easy to remember.

Detailed answer + Example - the real technical answer, with commands / YAML / code you can show.

What to say to the interviewer (blue box) - 2-3 lines to say out loud. Practise these aloud.

Questions that were asked more than once across the lists were merged into one (the other wordings are shown in grey as 'Also asked as'). Language-specific (Python/PHP) and non-DevOps questions were removed. Where a pasted 'model answer' was wrong, the corrected version is given.

Suggested order: all Basic questions first, then Scenario-Based (most interviews are troubleshooting-heavy), then Advanced. Use the PDF bookmarks panel to jump to any question.

Topic	Basic	Advanced	Scenario	Total
Shell Scripting: 24 Real Interview Scripts	7	8	9	24

Questions in this topic

Basic **4**

Q1. Write a script to clean files in /tmp that are older than 7 days. 4

Q2. Write a script to validate the IP addresses in a text file. 5

Q3. Write a script that reads server IPs from a file and pings each one. 6

Q4. Write a script to parse a log file and count failed login attempts. 7

Q5. Write a script to report all users with UID greater than 1000. 8

Q6. Write a script to compare two directories and show the differences. 9

Q7. Write a script to validate YAML and JSON files (for example in a CI pipeline). 10

Advanced **11**

Q8. Write a script to back up a MySQL database daily with a timestamp. 11

Q9. Write a script to SSH into multiple servers and run a command on each. 12

Q10. Write a script to archive application logs to S3. 13

Q11. Write a script to monitor SSL certificate expiry and alert before it expires. 15

Q12. Write a script to create Linux users from a CSV file. 16

Q13. Write a script to automate git pull and restart a service. 18

Q14. Write a script to generate a dynamic inventory file for Ansible. 19

Q15. Write a script that checks Jenkins job status through the API and alerts if the build failed. 20

Scenario-Based **21**

Q16. Production needs an alert when any disk goes above 80%. Write the script. 21

Q17. Write a script to monitor a web service and alert on failure. 22

Q18. Write a script to rotate logs and keep only the last 5 files. 23

Q19. Write a script to fetch the top CPU-consuming processes and email them. 24

Q20. Write a script to tail multiple log files in parallel. 25

Q21. Write a script to find zombie processes and clean them up. 26

Q22. Write a script to monitor network bandwidth in real time. 27

Q23. Write a script that alerts if a Docker container is not running. 28

Q24. Write a script to auto-deploy a static website using rsync. 29

Shell Scripting: 24 Real Interview Scripts

Basic

Q1. Write a script to clean files in /tmp that are older than 7 days.

> **In simple words:** /tmp is like a desk that everyone dumps papers on. This script throws away papers nobody has touched for a week, so the desk (disk) never overflows.

Detailed answer

- `find` walks the directory and selects only regular files whose modification time is older than 7 days (`-mtime +7`).
- A **dry run** mode prints what would be deleted before anything is removed - interviewers love this.
- `-xdev` stops find from crossing into other mounted filesystems; `-mindepth 1` protects the /tmp directory itself.
- Empty directories left behind are removed in a second pass.

Example:

```
#!/usr/bin/env bash
# clean_tmp.sh - delete files older than N days (default 7) from a directory
# Usage: clean_tmp.sh [-n] [dir] [days]     -n = dry run
set -euo pipefail

DRY_RUN=0
if [[ "${1:-}" == "-n" ]]; then DRY_RUN=1; shift; fi
DIR="${1:-/tmp}"
DAYS="${2:-7}"

[[ -d "$DIR" ]] || { echo "ERROR: $DIR is not a directory" >&2; exit 1; }
[[ "$DAYS" =~ ^[0-9]+$ ]] || { echo "ERROR: days must be an integer" >&2; exit 1; }
[[ "$DIR" == "/" ]] && { echo "ERROR: refusing to clean /" >&2; exit 1; }

if (( DRY_RUN )); then
  echo "Dry run - files that would be deleted:"
  find "$DIR" -xdev -mindepth 1 -type f -mtime +"$DAYS" -print
else
  count=$(find "$DIR" -xdev -mindepth 1 -type f -mtime +"$DAYS" -print -delete | wc -l)
  # remove empty directories left behind (but never DIR itself)
  find "$DIR" -xdev -mindepth 1 -type d -empty -mtime +"$DAYS" -delete
  logger -t clean_tmp "Deleted $count files older than $DAYS days from $DIR"
  echo "Deleted $count files"
fi
```

```
# cron: every day at 03:00
0 3 * * * /usr/local/bin/clean_tmp.sh /tmp 7 >/dev/null 2>&1
```

Watch out: Files still held open by a running process will vanish from the directory but not free space until the process closes them. On systemd machines, `systemd-tmpfiles` (`/etc/tmpfiles.d/`) already does this job - mention that as the built-in alternative.

> ***What to say to the interviewer:*** *I use* `find` *with* `-type f -mtime +7 -delete`*, add* `-xdev` *so it never crosses into other mounts, and always support a dry-run flag so I can see what will be*

removed first. I validate the inputs, refuse dangerous paths like `/`, log the count, and schedule it with cron - or just use systemd-tmpfiles, which is built for this.

Q2. Write a script to validate the IP addresses in a text file.

In simple words: Like checking that every phone number in a list has the right number of digits. An IPv4 address must be four numbers from 0 to 255 separated by dots - anything else is flagged.

Detailed answer

- A regex alone (`[0-9]{1,3}`) is not enough: it accepts `999.1.1.1`. So we check the **shape** with a regex, then check each **octet** is 0-255.
- Read the file line by line with `while IFS= read -r`, skipping blank lines and comments.
- Leading zeros like `010` are rejected, because some tools read them as octal.
- Exit code is non-zero if any invalid line was found, so the script can gate a CI job.

Example:

```
#!/usr/bin/env bash
# validate_ips.sh - report valid/invalid IPv4 addresses, one per line
set -uo pipefail

FILE="${1:?Usage: $0 <file>}"
[[ -r "$FILE" ]] || { echo "ERROR: cannot read $FILE" >&2; exit 2; }

is_valid_ip() {
  local ip="$1" octet
  [[ "$ip" =~ ^([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})$ ]] || return 1
  for octet in "${BASH_REMATCH[@]:1}"; do
    [[ "$octet" =~ ^(0|[1-9][0-9]*)$ ]] || return 1    # no leading zeros
    (( octet <= 255 )) || return 1
  done
  return 0
}

invalid=0; lineno=0
while IFS= read -r line || [[ -n "$line" ]]; do
  lineno=$((lineno + 1))
  line="${line//[[:space:]]/}"                          # trim spaces / CR
  [[ -z "$line" || "$line" == \#* ]] && continue
  if is_valid_ip "$line"; then
    echo "VALID    $line"
  else
    echo "INVALID  $line (line $lineno)"
    invalid=$((invalid + 1))
  fi
done < "$FILE"

echo "Invalid entries: $invalid"
(( invalid == 0 ))
```

```
$ ./validate_ips.sh ips.txt
VALID    10.0.1.15
INVALID  256.1.1.1 (line 2)
INVALID  192.168.1 (line 3)
VALID    8.8.8.8
Invalid entries: 2
```

Watch out: `|| [[ -n "$line" ]]` handles a last line with no trailing newline. Windows files carry `\r` - the whitespace strip removes it. For IPv6 or CIDR, mention `ipcalc` or a proper library rather than a monster regex.

> ***What to say to the interviewer:*** *I read the file line by line, match the four-octet shape with a Bash regex, then check every octet is between 0 and 255 with no leading zeros, because a regex alone would accept 999. The script prints valid and invalid lines with line numbers and exits non-zero if anything is invalid, so it can fail a pipeline.*

Q3. Write a script that reads server IPs from a file and pings each one.

> **In simple words:** A roll call for servers - read the list of names and shout each one; whoever answers is present (UP), whoever stays silent is marked absent (DOWN).

Detailed answer

- `ping -c 2 -W 2` sends 2 packets and waits at most 2 seconds, so a dead host does not hang the script.
- The sequential loop is simple; the **parallel** version uses `xargs -P 20` to ping 20 hosts at once - huge time saver with 500 servers.
- Results go to stdout and a summary; exit code = number of down hosts (capped), useful for monitoring.

Example:

```
#!/usr/bin/env bash
# ping_hosts.sh - ping every host listed in a file (sequential or parallel)
# Usage: ping_hosts.sh hosts.txt [parallelism]
set -uo pipefail

FILE="${1:?Usage: $0 <hosts_file> [parallel]}"
PARALLEL="${2:-1}"
[[ -r "$FILE" ]] || { echo "ERROR: cannot read $FILE" >&2; exit 2; }

check_host() {
  local host="$1"
  if ping -c 2 -W 2 "$host" >/dev/null 2>&1; then
    echo "UP    $host"
  else
    echo "DOWN  $host"
  fi
}
export -f check_host

grep -Ev '^[[:space:]]*(#|$)' "$FILE" \
  | xargs -r -n1 -P "$PARALLEL" bash -c 'check_host "$1"' _ \
  | tee /tmp/ping_results.txt

down=$(grep -c '^DOWN' /tmp/ping_results.txt || true)
echo "Summary: $down host(s) down"
exit $(( down > 125 ? 125 : down ))
```

```
$ ./ping_hosts.sh servers.txt 20
UP    10.0.1.10
DOWN  10.0.1.11
UP    10.0.2.20
Summary: 1 host(s) down
```

Watch out: Many cloud hosts block ICMP in the security group, so "no ping" does not always mean "down" - for real health use a TCP check like `nc -z -w2 host 22`. Also `-W` is Linux ping syntax; macOS uses different flags.

> ***What to say to the interviewer:*** *I read the file skipping comments and blanks, ping each host with a count and a timeout so nothing hangs, and print UP or DOWN. For big lists I run it in parallel with* `xargs -P`*, and I point out that ICMP can be blocked, so for real health checks I would use a TCP port check with* `nc`*.*

Q4. Write a script to parse a log file and count failed login attempts.

> **In simple words:** Like a security guard reading the visitor book and counting how many times each stranger tried the wrong key - the top names are probably attackers.

Detailed answer

- SSH failures appear in `/var/log/auth.log` (Debian/Ubuntu) or `/var/log/secure` (RHEL) as `Failed password for ... from <IP>`.
- `grep` finds the lines, awk/`grep -oE` extracts the IP, then `sort | uniq -c | sort -nr` counts and ranks them.
- Also report the targeted usernames, and flag IPs above a threshold (these are candidates for fail2ban or a WAF/SG block).

Example:

```
#!/usr/bin/env bash
# failed_logins.sh - count failed SSH logins per IP and per user
set -euo pipefail

LOG="${1:-}"
if [[ -z "$LOG" ]]; then
  [[ -f /var/log/auth.log ]] && LOG=/var/log/auth.log || LOG=/var/log/secure
fi
THRESHOLD="${2:-10}"
[[ -r "$LOG" ]] || { echo "ERROR: cannot read $LOG (need sudo?)" >&2; exit 1; }

total=$(grep -c 'Failed password' "$LOG" || true)
echo "Total failed password attempts: $total"

echo; echo "Top source IPs:"
grep 'Failed password' "$LOG" \
  | grep -oE 'from [0-9]{1,3}(\.[0-9]{1,3}){3}' | awk '{print $2}' \
  | sort | uniq -c | sort -nr | head -10

echo; echo "Top targeted users:"
grep 'Failed password' "$LOG" \
  | sed -E 's/.*Failed password for (invalid user )?([^ ]+) from.*/\2/' \
  | sort | uniq -c | sort -nr | head -10

echo; echo "IPs over threshold ($THRESHOLD):"
grep 'Failed password' "$LOG" \
  | grep -oE 'from [0-9]{1,3}(\.[0-9]{1,3}){3}' | awk '{print $2}' \
  | sort | uniq -c | awk -v t="$THRESHOLD" '$1 > t {print $2, "(" $1 " attempts)"}'
```

```
# systemd-only systems keep logs in the journal instead of a file:
journalctl -u ssh --since "24 hours ago" | grep 'Failed password' | wc -l
```

Watch out: Rotated logs (`auth.log.1`, `auth.log.2.gz`) hold older attempts - use `zgrep` to include them. The sed handles the "invalid user" variant, which otherwise breaks naive `awk '{print $9}'` column counting.

> ***What to say to the interviewer:*** *I grep for "Failed password" in auth.log or secure, extract the source IP with* `grep -o`*, and use* `sort | uniq -c | sort -nr` *to rank attackers, plus a second pass for targeted usernames. Anything over a threshold gets flagged for fail2ban or a firewall block, and I use zgrep or journalctl when logs are rotated or in the journal.*

Q5. Write a script to report all users with UID greater than 1000.

> **In simple words:** Linux keeps a guest list in `/etc/passwd`. System accounts get low numbers; real people usually start at 1000. This script prints the real people.

Detailed answer

- `/etc/passwd` is colon-separated: `name:x:UID:GID:comment:home:shell`, so `awk -F:` reads field 3 as the UID.
- The question says "> 1000", but normal users start **at** 1000 on most distros (`UID_MIN` in `/etc/login.defs`), so use `>= 1000` and say why.
- Exclude `nobody` (UID 65534), which is a system account despite the big number.
- `getent passwd` is better than reading the file because it also includes LDAP/SSSD users.

Example:

```
#!/usr/bin/env bash
# human_users.sh - list regular (human) user accounts
set -euo pipefail

MIN_UID=$(awk '/^UID_MIN/ {print $2}' /etc/login.defs 2>/dev/null || true)
MIN_UID="${MIN_UID:-1000}"

printf "%-15s %-7s %-25s %s\n" "USER" "UID" "HOME" "SHELL"
getent passwd | awk -F: -v min="$MIN_UID" \
  '$3 >= min && $3 != 65534 { printf "%-15s %-7s %-25s %s\n", $1, $3, $6, $7 }' \
  | sort -k2 -n
```

```
$ ./human_users.sh
USER            UID     HOME                      SHELL
priyanka        1000    /home/priyanka            /bin/bash
deploy          1001    /home/deploy              /bin/bash

# The literal one-liner an interviewer expects:
awk -F: '$3 > 1000 {print $1, $3}' /etc/passwd
```

Watch out: Mention the > vs >= detail - it shows you know UID_MIN. Accounts with `/usr/sbin/nologin` shells are service users even with high UIDs; you can filter on the shell field too.

> ***What to say to the interviewer:*** *The quick answer is* `awk -F: '$3 > 1000' /etc/passwd`*, but I use* `getent passwd` *so directory users are included, read UID_MIN from login.defs because human users start at 1000, and exclude nobody at 65534. I print user, UID, home and shell in a table sorted by UID.*

Q6. Write a script to compare two directories and show the differences.

> **In simple words:** Like comparing two photo albums side by side - which photos are only in the left album, which only in the right, and which have the same name but a different picture.

Detailed answer

- `diff -rq dir1 dir2` is the one-liner: recursive, quiet (names only).
- `rsync -rcn --delete --itemize-changes src/ dst/` is a dry run that shows exactly what would change if you synced them - `-c` compares checksums, not just size and time.
- For large trees, a checksum manifest (`sha256sum`) is reliable and can be compared across servers.

Example:

```
#!/usr/bin/env bash
# dircompare.sh - compare two directory trees by content
set -euo pipefail

A="${1:?Usage: $0 <dir1> <dir2>}"
B="${2:?Usage: $0 <dir1> <dir2>}"
for d in "$A" "$B"; do [[ -d "$d" ]] || { echo "ERROR: $d not a dir" >&2; exit 2; }; done

manifest() {   # relative path + sha256 for every file, NUL-safe
  (cd "$1" && find . -type f -print0 | sort -z | xargs -0 -r sha256sum)
}

tmpA=$(mktemp); tmpB=$(mktemp)
trap 'rm -f "$tmpA" "$tmpB"' EXIT
manifest "$A" > "$tmpA"
manifest "$B" > "$tmpB"

echo "== Only in $A =="
comm -23 <(awk '{print $2}' "$tmpA") <(awk '{print $2}' "$tmpB")
echo "== Only in $B =="
comm -13 <(awk '{print $2}' "$tmpA") <(awk '{print $2}' "$tmpB")
echo "== Same name, different content =="
join -j 2 <(sort -k2 "$tmpA") <(sort -k2 "$tmpB") | awk '$2 != $3 {print $1}'

if cmp -s "$tmpA" "$tmpB"; then echo "RESULT: identical"; exit 0; fi
echo "RESULT: different"; exit 1
```

```
# Quick alternatives
diff -rq /srv/app/releases/v1 /srv/app/releases/v2
rsync -rcn --delete --itemize-changes /data/src/ backup-host:/data/dst/
```

Watch out: The manifest uses `awk '{print $2}'`, which breaks on filenames containing spaces - for those, rely on `diff -rq` or `rsync -n`. Always put the trailing slash on rsync source dirs (`src/`) or you compare the wrong level.

> ***What to say to the interviewer:*** *For a quick check I use* `diff -rq` *or an* `rsync -rcn --delete` *dry run, which compares by checksum. For a full report I build a sha256 manifest of each tree and use* `comm` *and* `join` *to list files only on one side and files whose content differs, and the exit code tells a pipeline whether they match.*

Q7. Write a script to validate YAML and JSON files (for example in a CI pipeline).

In simple words: A spell-checker for config files. One missing bracket or a bad indent in YAML can take down a deployment, so we check every file before it ships.

Detailed answer

- JSON: `jq empty file.json` parses the file and prints nothing if it is valid; non-zero exit if broken.
- YAML: `yq eval '.' file.yaml >/dev/null` checks syntax; `yamllint` also checks style (indentation, duplicate keys, trailing spaces).
- Loop over all matching files with `find -print0`, collect failures, exit non-zero if any failed - this makes it a CI gate.

Example:

```
#!/usr/bin/env bash
# validate_configs.sh - syntax-check all JSON and YAML files under a directory
set -uo pipefail

ROOT="${1:-.}"
fail=0; checked=0

for tool in jq yq; do
  command -v "$tool" >/dev/null || { echo "ERROR: $tool not installed" >&2; exit 2; }
done

while IFS= read -r -d '' f; do
  checked=$((checked + 1))
  case "$f" in
    *.json)       jq empty "$f" 2>/tmp/val_err ;;
    *.yaml|*.yml) yq eval '.' "$f" >/dev/null 2>/tmp/val_err ;;
  esac
  if [[ $? -ne 0 ]]; then
    echo "FAIL  $f: $(head -1 /tmp/val_err)"
    fail=$((fail + 1))
  else
    echo "OK    $f"
  fi
done < <(find "$ROOT" -type f \( -name '*.json' -o -name '*.yaml' -o -name '*.yml' \) \
          -not -path '*/node_modules/*' -not -path '*/.git/*' -print0)

if command -v yamllint >/dev/null; then
  yamllint -d relaxed "$ROOT" || fail=$((fail + 1))
fi

echo "Checked $checked files, $fail failure(s)"
(( fail == 0 ))
```

```
# GitHub Actions step
- name: Validate configs
  run: ./scripts/validate_configs.sh k8s/
```

Watch out: Valid YAML is not the same as a valid Kubernetes manifest - add `kubeconform` or `kubectl apply --dry-run=server` for schema checks. Helm templates (`{{ }}`) are not valid YAML until rendered, so run `helm template` first or exclude chart templates.

What to say to the interviewer: *I loop over every JSON and YAML file with a NUL-safe find, validate JSON with* `jq empty` *and YAML with* `yq`*, optionally run yamllint for style, and exit non-zero*

if anything fails so the CI job blocks the merge. For Kubernetes I go one step further with kubeconform, because syntactically valid YAML can still be an invalid manifest.

Advanced

Q8. Write a script to back up a MySQL database daily with a timestamp.

In simple words: A daily photograph of the database, labelled with the date, kept for a set number of days - if something breaks, you restore yesterday's photo.

Detailed answer

- `mysqldump --single-transaction` takes a consistent snapshot of InnoDB tables **without locking** them - important for production.
- Credentials come from a protected `~/.my.cnf` (mode 600), **never** `-pPassword` on the command line, which shows up in `ps` and shell history.
- Output is piped to `gzip` with a timestamped filename; `set -o pipefail` makes the script fail if mysqldump fails mid-pipe.
- Verify the backup (gzip integrity + dump completion marker), apply retention, and ideally copy it off-server (S3).

Example:

```
#!/usr/bin/env bash
# mysql_backup.sh - daily compressed MySQL backup with retention
set -euo pipefail

DB="${1:?Usage: $0 <database> [retention_days]}"
RETENTION="${2:-7}"
BACKUP_DIR="/var/backups/mysql"
TS=$(date +%Y%m%d_%H%M%S)
OUT="$BACKUP_DIR/${DB}_${TS}.sql.gz"
DEFAULTS="/root/.my.cnf"            # [client] user=... password=... (chmod 600)

mkdir -p "$BACKUP_DIR"
umask 077                           # backups readable by owner only

log() { logger -t mysql_backup "$*"; echo "$(date '+%F %T') $*"; }

log "Starting backup of $DB -> $OUT"
mysqldump --defaults-extra-file="$DEFAULTS" \
  --single-transaction --quick --routines --triggers --events \
  "$DB" | gzip > "$OUT"

# verify: archive not corrupt and dump finished
gzip -t "$OUT"
zcat "$OUT" | tail -1 | grep -q 'Dump completed' \
  || { log "ERROR: dump incomplete"; rm -f "$OUT"; exit 1; }

log "Backup OK: $(du -h "$OUT" | cut -f1)"

# optional off-site copy (EC2 IAM role provides credentials)
# aws s3 cp "$OUT" "s3://my-db-backups/$DB/" --sse aws:kms

find "$BACKUP_DIR" -name "${DB}_*.sql.gz" -type f -mtime +"$RETENTION" -delete
log "Removed backups older than $RETENTION days"
```

```
# cron: every day at 01:30
30 1 * * * /usr/local/bin/mysql_backup.sh appdb 7 >> /var/log/mysql_backup.log 2>&1

# restore
zcat /var/backups/mysql/appdb_20260924_013000.sql.gz | mysql appdb
```

Watch out: A backup you have never restored is not a backup - test restores regularly. `--single-transaction` only gives consistency for InnoDB; MyISAM tables still need locks. On RDS, prefer automated snapshots and PITR, and use a dump only for logical exports.

> ***What to say to the interviewer:*** *I use* `mysqldump --single-transaction` *so InnoDB is dumped consistently without locking, read credentials from a chmod-600 .my.cnf instead of the command line, and pipe to gzip with a timestamped filename under pipefail. Then I verify the archive, apply retention with find, copy it to S3 with encryption, and schedule it in cron - and I test restores regularly.*

Q9. Write a script to SSH into multiple servers and run a command on each.

> **In simple words:** Instead of logging into 50 servers one by one to type the same command, the script knocks on every door for you and collects the answers in one place.

Detailed answer

- `-o BatchMode=yes` makes SSH fail immediately instead of prompting for a password - essential for automation.
- `-o ConnectTimeout=5` stops dead hosts from hanging the run; `StrictHostKeyChecking=accept-new` accepts new hosts but still rejects changed keys.
- Run in parallel with background jobs or `xargs -P`, prefix every output line with the hostname, and collect a per-host exit status.
- Mention that for real fleets **Ansible ad-hoc** (`ansible all -m shell -a "uptime"`) is the better tool.

Example:

```
#!/usr/bin/env bash
# multi_ssh.sh - run a command on many hosts in parallel
# Usage: multi_ssh.sh hosts.txt "uptime" [parallel]
set -uo pipefail

HOSTS="${1:?Usage: $0 <hosts_file> <command> [parallel]}"
CMD="${2:?Usage: $0 <hosts_file> <command> [parallel]}"
PARALLEL="${3:-10}"
SSH_USER="${SSH_USER:-ec2-user}"
LOGDIR=$(mktemp -d)

run_on_host() {
  local host="$1"
  if ssh -n -o BatchMode=yes -o ConnectTimeout=5 \
         -o StrictHostKeyChecking=accept-new \
         "$SSH_USER@$host" "$CMD" > "$LOGDIR/$host.out" 2>&1; then
    echo "OK $host" >> "$LOGDIR/status"
  else
    echo "FAIL $host (exit $?)" >> "$LOGDIR/status"
  fi
  sed "s/^/[$host] /" "$LOGDIR/$host.out"
}
export -f run_on_host
export CMD SSH_USER LOGDIR

grep -Ev '^[[:space:]]*(#|$)' "$HOSTS" | xargs -r -n1 -P "$PARALLEL" bash -c 'run_on_host "$1"' _

echo "---- Summary ----"
sort "$LOGDIR/status"
failed=$(grep -c '^FAIL' "$LOGDIR/status" || true)
rm -rf "$LOGDIR"
exit $(( failed > 0 ? 1 : 0 ))
```

```
$ SSH_USER=ubuntu ./multi_ssh.sh web_servers.txt "df -h / | tail -1" 20
[10.0.1.10] /dev/nvme0n1p1   30G   21G  9.0G  70% /
[10.0.1.11] /dev/nvme0n1p1   30G   28G  2.0G  94% /
---- Summary ----
OK 10.0.1.10
OK 10.0.1.11

# The better tool for fleets:
ansible webservers -i inventory.ini -m shell -a "df -h /" -f 20
```

Watch out: `ssh -n` stops SSH from eating the rest of the host list from stdin - a classic bug in `while read` loops. Never run destructive commands across all hosts without a dry run or a canary host first. On AWS, SSM Run Command does this without opening port 22 at all.

> ***What to say to the interviewer:*** *I read the host list, run SSH with BatchMode and ConnectTimeout so it never prompts or hangs, execute in parallel with* `xargs -P`*, prefix each output line with the hostname and print a per-host OK/FAIL summary. For a real fleet I would use Ansible ad-hoc commands or AWS SSM Run Command, which handle inventory, parallelism and auditing for me.*

Q10. Write a script to archive application logs to S3.

> **In simple words:** Old logs are like old files in an office drawer - you pack them into a labelled box, send the box to cheap long-term storage (S3), and only then clear the drawer.

Detailed answer

- Find log files older than N days, bundle them into a timestamped `tar.gz`.
- Upload with `aws s3 cp --sse aws:kms` under a date/host prefix, e.g. `s3://bucket/app/2026/09/24/host/`.
- **Delete local files only after the upload succeeds** (check the exit code, optionally compare sizes).
- Credentials come from the **EC2 IAM instance role**, not access keys in the script. An S3 lifecycle rule moves old archives to Glacier.

Example:

```
#!/usr/bin/env bash
# archive_logs_s3.sh - compress logs older than N days and ship them to S3
set -euo pipefail

LOG_DIR="${1:-/var/log/myapp}"
BUCKET="${2:?Usage: $0 <log_dir> <s3_bucket> [days]}"
DAYS="${3:-3}"
HOST=$(hostname -s)
TS=$(date +%Y%m%d_%H%M%S)
PREFIX="myapp/$(date +%Y/%m/%d)/$HOST"
ARCHIVE="/tmp/${HOST}_logs_${TS}.tar.gz"
LIST=$(mktemp)
trap 'rm -f "$LIST" "$ARCHIVE"' EXIT

find "$LOG_DIR" -type f -name '*.log*' -mtime +"$DAYS" -print0 > "$LIST"
if [[ ! -s "$LIST" ]]; then echo "Nothing to archive"; exit 0; fi

tar -czf "$ARCHIVE" --null -T "$LIST"
echo "Created $ARCHIVE ($(du -h "$ARCHIVE" | cut -f1))"

if aws s3 cp "$ARCHIVE" "s3://$BUCKET/$PREFIX/" --sse aws:kms --only-show-errors; then
  # confirm the object exists before deleting anything locally
  aws s3api head-object --bucket "$BUCKET" --key "$PREFIX/$(basename "$ARCHIVE")" >/dev/null
  xargs -0 -r rm -f < "$LIST"
  logger -t archive_logs "Archived and removed $(tr -cd '\0' < "$LIST" | wc -c) files"
else
  echo "ERROR: upload failed - local logs kept" >&2
  exit 1
fi
```

```
# cron: daily at 02:00
0 2 * * * /usr/local/bin/archive_logs_s3.sh /var/log/myapp my-log-archive 3
```

```
{ "Effect": "Allow", "Action": ["s3:PutObject"],
  "Resource": "arn:aws:s3:::my-log-archive/myapp/*" }
```

Watch out: Do not archive the file the app is currently writing to - coordinate with logrotate (rotate first, archive rotated files). The IAM role needs only `s3:PutObject` on that prefix, plus `kms:GenerateDataKey` for SSE-KMS. For continuous shipping, a log agent (Fluent Bit / CloudWatch agent) is better than a cron script.

> ***What to say to the interviewer:*** *I find logs older than N days with a NUL-safe find, tar and gzip them with a host and timestamp name, upload with* `aws s3 cp --sse aws:kms` *under a date/host prefix using the instance role, and delete the local files only after I confirm the object exists in S3. A lifecycle rule then tiers the archives to Glacier to keep cost low.*

Q11. Write a script to monitor SSL certificate expiry and alert before it expires.

> **In simple words:** Like a reminder for your passport's expiry date - the script checks each website's certificate and warns you 30 days before it runs out, so customers never see a "Not Secure" page.

Detailed answer

- `openssl s_client -connect host:443 -servername host` fetches the live certificate (`-servername` sends SNI, required for most modern hosts).
- `openssl x509 -noout -enddate` prints `notAfter=...`; convert it to epoch seconds with `date -d` and compute days left.
- Loop over a list of domains; warn under 30 days, critical under 7, and post to a Teams/Slack webhook.

Example:

```
#!/usr/bin/env bash
# ssl_expiry.sh - check certificate expiry for a list of domains
# Usage: ssl_expiry.sh domains.txt   (lines: domain[:port])
set -uo pipefail

FILE="${1:?Usage: $0 <domains_file>}"
WARN_DAYS="${WARN_DAYS:-30}"
CRIT_DAYS="${CRIT_DAYS:-7}"
WEBHOOK="${WEBHOOK_URL:-}"          # Teams/Slack incoming webhook (optional)
status=0

alert() {
  echo "$1"
  [[ -n "$WEBHOOK" ]] && curl -s -m 10 -H 'Content-Type: application/json' \
    -d "{\"text\": \"$1\"}" "$WEBHOOK" >/dev/null
}

while IFS= read -r entry; do
  [[ -z "$entry" || "$entry" == \#* ]] && continue
  host="${entry%%:*}"; port="${entry##*:}"; [[ "$port" == "$host" ]] && port=443

  end=$(echo | timeout 10 openssl s_client -connect "$host:$port" -servername "$host" \
        2>/dev/null | openssl x509 -noout -enddate 2>/dev/null | cut -d= -f2)
  if [[ -z "$end" ]]; then
    alert "UNKNOWN: could not read certificate for $host:$port"; status=2; continue
  fi

  days=$(( ( $(date -d "$end" +%s) - $(date +%s) ) / 86400 ))
  if (( days < CRIT_DAYS )); then
    alert "CRITICAL: $host cert expires in $days days ($end)"; status=2
  elif (( days < WARN_DAYS )); then
    alert "WARNING: $host cert expires in $days days ($end)"; (( status < 1 )) && status=1
  else
    echo "OK: $host - $days days left"
  fi
done < "$FILE"
exit "$status"
```

```
$ ./ssl_expiry.sh domains.txt
OK: api.example.com - 74 days left
WARNING: portal.example.com cert expires in 18 days (Oct 12 23:59:59 2026 GMT)

# daily at 09:00
0 9 * * * WEBHOOK_URL=https://... /usr/local/bin/ssl_expiry.sh /etc/ssl_domains.txt
```

Watch out: Without `-servername` you may get the default certificate of the load balancer, not the site's. `date -d` is GNU; on macOS use `date -j -f`. ACM certificates auto-renew, but imported certificates do not - this script catches exactly those. Blackbox exporter (`probe_ssl_earliest_cert_expiry`) is the Prometheus way.

> ***What to say to the interviewer:*** *I pull the live certificate with* `openssl s_client` *using SNI, read notAfter with* `openssl x509 -enddate`*, convert it to epoch to get days left, and alert to Teams under 30 days and critically under 7, with Nagios-style exit codes. In a Prometheus setup I would use the blackbox exporter's cert expiry metric instead of a cron script.*

Q12. Write a script to create Linux users from a CSV file.

> **In simple words:** HR hands you a spreadsheet of new joiners. Instead of creating 40 accounts by hand, the script reads each row and creates the user, their group and a temporary password - and skips anyone who already exists.

Detailed answer

- CSV format: `username,full_name,group,shell`. Skip the header, validate each field.
- **Idempotent**: if the user already exists, skip it (running the script twice must not break anything).
- `useradd -m -c -s -G` creates home, comment, shell and supplementary group; `chpasswd` sets a random temporary password; `chage -d 0` forces a change at first login.
- Must run as root; log every action; write the temporary passwords to a root-only file (better: SSH keys only, no passwords).

Example:

```
#!/usr/bin/env bash
# create_users.sh - bulk create users from CSV: username,full_name,group,shell
set -euo pipefail

CSV="${1:?Usage: $0 <users.csv>}"
[[ $EUID -eq 0 ]] || { echo "ERROR: run as root" >&2; exit 1; }
[[ -r "$CSV" ]] || { echo "ERROR: cannot read $CSV" >&2; exit 1; }

CREDS="/root/new_user_passwords_$(date +%F).txt"
umask 077
created=0; skipped=0

while IFS=, read -r user name group shell || [[ -n "$user" ]]; do
  user=$(echo "$user" | tr -d '[:space:]')
  [[ -z "$user" || "$user" == "username" || "$user" == \#* ]] && continue
  shell="${shell//$'\r'/}"; shell="${shell:-/bin/bash}"

  if [[ ! "$user" =~ ^[a-z_][a-z0-9_-]{0,31}$ ]]; then
    echo "SKIP invalid username: $user"; skipped=$((skipped + 1)); continue
  fi
  if id "$user" &>/dev/null; then
    echo "SKIP exists: $user"; skipped=$((skipped + 1)); continue
  fi
  if [[ -n "$group" ]] && ! getent group "$group" >/dev/null; then
    groupadd "$group" && echo "Created group $group"
  fi

  useradd -m -c "$name" -s "$shell" ${group:+-G "$group"} "$user"
  pass=$(openssl rand -base64 12)
  echo "$user:$pass" | chpasswd
  chage -d 0 "$user"                 # force password change at first login
  echo "$user,$pass" >> "$CREDS"
  echo "CREATED $user (group: ${group:-none})"
  created=$((created + 1))
done < "$CSV"

echo "Done: $created created, $skipped skipped. Temp passwords in $CREDS"
```

```
$ cat users.csv
username,full_name,group,shell
asha,Asha Rao,devops,/bin/bash
ravi,Ravi Kumar,developers,/bin/bash
$ sudo ./create_users.sh users.csv
Created group devops
CREATED asha (group: devops)
SKIP exists: ravi
```

Watch out: Never echo passwords to the terminal or logs, and delete the credentials file after handing them over. Commas inside names break a naive CSV split - keep the format simple or quote-aware. At scale, users should come from LDAP/SSO or Ansible's `user` module, not a local script.

> ***What to say to the interviewer:*** *I read the CSV with* `IFS=,` *, skip the header, validate the username, and make it idempotent by skipping users that already exist. I create missing groups, run* `useradd -m` *with the shell and group, set a random temporary password with chpasswd and force a change at first login with* `chage -d 0`*, and log everything. In a real company I would push this into Ansible or SSO rather than local accounts.*

Q13. Write a script to automate git pull and restart a service.

> **In simple words:** A simple self-updating server: it checks whether new code has arrived on the main branch, and only if it has, it pulls it and restarts the app - and puts the old version back if the new one fails its health check.

Detailed answer

- `git fetch` then compare local `HEAD` with `origin/main`; restart **only when there is a change**.
- `git reset --hard origin/main` (instead of `git pull`) avoids merge conflicts from local edits on the server.
- `flock` prevents two cron runs from overlapping.
- After the restart, run a health check; on failure, reset to the previous commit and restart again (rollback).

Example:

```
#!/usr/bin/env bash
# git_deploy.sh - pull latest main and restart service only when code changed
set -euo pipefail

APP_DIR="/opt/myapp"
BRANCH="main"
SERVICE="myapp"
HEALTH_URL="http://localhost:8080/health"

exec 9>/var/lock/git_deploy.lock
flock -n 9 || { echo "Another deploy is running"; exit 0; }

log() { logger -t git_deploy "$*"; echo "$(date '+%F %T') $*"; }
cd "$APP_DIR"

git fetch --quiet origin "$BRANCH"
LOCAL=$(git rev-parse HEAD)
REMOTE=$(git rev-parse "origin/$BRANCH")
if [[ "$LOCAL" == "$REMOTE" ]]; then
  log "No changes ($LOCAL)"; exit 0
fi

log "Updating $LOCAL -> $REMOTE"
git reset --hard "origin/$BRANCH"
systemctl restart "$SERVICE"

for i in {1..10}; do
  if curl -fsS -m 3 "$HEALTH_URL" >/dev/null; then
    log "Deploy OK at $REMOTE"; exit 0
  fi
  sleep 3
done

log "Health check failed - rolling back to $LOCAL"
git reset --hard "$LOCAL"
systemctl restart "$SERVICE"
exit 1
```

```
# check for new commits every 5 minutes
*/5 * * * * /usr/local/bin/git_deploy.sh >> /var/log/git_deploy.log 2>&1
```

Watch out: `reset --hard` destroys any local edits on the server - that is intentional (servers should be read-only copies), but say it out loud. Use a deploy key with read-only access. This is fine for a small internal app; for production the better pattern is a CI/CD pipeline that builds an immutable artifact, or GitOps with Argo CD.

> ***What to say to the interviewer:*** *I fetch from origin, compare HEAD with origin/main and exit early if nothing changed, then* `reset --hard` *to avoid merge conflicts, restart the service and poll a health endpoint. If the health check fails I reset to the previous commit and restart again, and I wrap the whole thing in flock so cron runs never overlap. For real production I would move this to a pipeline or GitOps.*

Q14. Write a script to generate a dynamic inventory file for Ansible.

> **In simple words:** In the cloud, servers appear and disappear all day, so a hand-written list of servers goes stale. This script asks AWS "which servers are running right now with this tag?" and builds Ansible's list fresh every time.

Detailed answer

- `aws ec2 describe-instances` filtered by tag and state `running`, with a `--query` to pull private IP and Role tag.
- Ansible dynamic inventory scripts must print JSON when called with `--list` (and handle `--host`).
- Group hosts by their `Role` tag, e.g. `web`, `db`.
- Mention the **proper way**: the `amazon.aws.aws_ec2` inventory plugin does this natively with caching and keyed groups.

Example:

```
#!/usr/bin/env bash
# ec2_inventory.sh - Ansible dynamic inventory from EC2 tags
# Usage: ansible-playbook -i ec2_inventory.sh site.yml
set -euo pipefail

ENV_TAG="${INV_ENV:-prod}"
REGION="${AWS_REGION:-ap-south-1}"

case "${1:-}" in
  --host) echo '{}'; exit 0 ;;           # hostvars are provided in _meta
  --list) ;;
  *) echo "Usage: $0 --list | --host <name>" >&2; exit 1 ;;
esac

aws ec2 describe-instances --region "$REGION" \
  --filters "Name=tag:Environment,Values=$ENV_TAG" "Name=instance-state-name,Values=running" \
  --query 'Reservations[].Instances[].{ip:PrivateIpAddress, role:Tags[?Key==`Role`]|[0].Value}' \
  --output json \
| jq '
    map(select(.ip != null) | .role = (.role // "ungrouped"))
    | reduce .[] as $i ({"_meta": {"hostvars": {}}};
        .[$i.role].hosts += [$i.ip]
        | ._meta.hostvars[$i.ip] = {"ansible_user": "ec2-user", "role": $i.role})
  '
```

```
$ ./ec2_inventory.sh --list
{
  "_meta": { "hostvars": { "10.0.1.10": { "ansible_user": "ec2-user", "role": "web" } } },
  "web": { "hosts": ["10.0.1.10", "10.0.1.11"] },
  "db":  { "hosts": ["10.0.2.20"] }
}
```

```
# The proper way - aws_ec2.yml inventory plugin
plugin: amazon.aws.aws_ec2
regions: [ap-south-1]
filters:
  tag:Environment: prod
  instance-state-name: running
keyed_groups:
  - key: tags.Role
    prefix: role
compose:
  ansible_host: private_ip_address
```

Watch out: The script must be executable and output only JSON (no echo debugging on stdout). Use the instance role or an SSO profile for AWS credentials. Private IPs need network reachability from the Ansible control node (VPN/bastion/SSM).

> ***What to say to the interviewer:*** *My script calls* `aws ec2 describe-instances` *filtered on the Environment tag and running state, then uses jq to group private IPs by their Role tag into the JSON format Ansible expects for* `--list`*, with hostvars in* `_meta`*. In practice I prefer the* `amazon.aws.aws_ec2` *inventory plugin, which does the same with keyed groups and caching and no custom code.*

Q15. Write a script that checks Jenkins job status through the API and alerts if the build failed.

> **In simple words:** Instead of refreshing the Jenkins page, the script asks Jenkins directly "how did the last build go?" and sends a message to the team channel if the answer is FAILURE or UNSTABLE.

Detailed answer

- Every Jenkins page has a JSON API: `$JENKINS_URL/job/<name>/lastBuild/api/json`.
- Authenticate with **username + API token** (not the password). A CSRF crumb is **not** needed for GET requests, only for POSTs.
- `jq -r .result` gives `SUCCESS`, `FAILURE`, `UNSTABLE`, `ABORTED`, or `null` while still building.
- Loop over several jobs; alert to a Teams/Slack webhook with the build URL; exit non-zero on failure.

Example:

```
#!/usr/bin/env bash
# jenkins_status.sh - alert when the last build of any listed job failed
# Usage: JENKINS_USER=.. JENKINS_TOKEN=.. jenkins_status.sh job1 folder/job/job2
set -uo pipefail

JENKINS_URL="${JENKINS_URL:-https://jenkins.example.com}"
: "${JENKINS_USER:?set JENKINS_USER}" "${JENKINS_TOKEN:?set JENKINS_TOKEN}"
WEBHOOK="${WEBHOOK_URL:-}"
[[ $# -gt 0 ]] || { echo "Usage: $0 <job> [job...]" >&2; exit 2; }
failed=0

notify() {
  echo "$1"
  [[ -n "$WEBHOOK" ]] && curl -s -m 10 -H 'Content-Type: application/json' \
    -d "$(jq -n --arg t "$1" '{text: $t}')" "$WEBHOOK" >/dev/null
}

for job in "$@"; do
  json=$(curl -fsS -m 15 -u "$JENKINS_USER:$JENKINS_TOKEN" \
         "$JENKINS_URL/job/$job/lastBuild/api/json?tree=result,number,url,building") \
    || { notify "ERROR: cannot query Jenkins job $job"; failed=1; continue; }

  result=$(jq -r '.result' <<< "$json")
  number=$(jq -r '.number' <<< "$json")
  url=$(jq -r '.url' <<< "$json")

  case "$result" in
    SUCCESS) echo "OK      $job #$number" ;;
    null)    echo "RUNNING $job #$number" ;;
    FAILURE|UNSTABLE)
      notify "Jenkins $job #$number is $result - $url"; failed=1 ;;
    *)       echo "INFO    $job #$number result=$result" ;;
  esac
done
exit "$failed"
```

```
$ ./jenkins_status.sh api-service ui-service
OK      api-service #412
Jenkins ui-service #97 is FAILURE - https://jenkins.example.com/job/ui-service/97/
```

Watch out: Jobs inside folders use the path `job/folder/job/name`. Keep the API token in a secret store or credentials file, not in the crontab. Using `?tree=` limits the JSON size, which matters on big Jenkins instances. Native alternatives: the `post { failure { ... } }` block in the Jenkinsfile, or the Office 365/Slack notification plugins.

> ***What to say to the interviewer:*** *I call the Jenkins JSON API at* `job/<name>/lastBuild/api/json` *with a user and API token, use jq to read the result, number and URL, and send a Teams or Slack message when the result is FAILURE or UNSTABLE. GET calls do not need a CSRF crumb, I use the tree parameter to keep responses small, and I note that a* `post { failure }` *block in the Jenkinsfile is the native way to do the same.*

Scenario-Based

Q16. Production needs an alert when any disk goes above 80%. Write the script.

> **In simple words:** A fuel gauge warning light for every disk. The script checks each filesystem and shouts on Teams or Slack before the tank is empty.

Detailed answer

- `df -P` gives POSIX output with one line per filesystem, so it never wraps and is safe to parse with awk.
- Skip pseudo filesystems (tmpfs, devtmpfs, overlay, squashfs) so you don't get false alarms.
- Strip the `%` sign and compare as an integer.
- Send one message listing every disk that crossed the threshold, and exit non-zero so cron or monitoring can see the failure.

Example:

```
#!/usr/bin/env bash
# disk_alert.sh - alert when any filesystem crosses a usage threshold
set -euo pipefail

THRESHOLD="${1:-80}"
WEBHOOK_URL="${WEBHOOK_URL:?set WEBHOOK_URL (Teams/Slack incoming webhook)}"
HOST="$(hostname -f)"

[[ "$THRESHOLD" =~ ^[0-9]+$ ]] || { echo "usage: $0 [threshold%]" >&2; exit 2; }

alerts="$(df -P -x tmpfs -x devtmpfs -x overlay -x squashfs \
  | awk -v t="$THRESHOLD" 'NR>1 { gsub("%","",$5); if ($5+0 >= t) \
      printf "%s is %s%% full (mount %s)\\n", $1, $5, $6 }')"

if [[ -n "$alerts" ]]; then
  msg="Disk alert on ${HOST} (threshold ${THRESHOLD}%):\n${alerts}"
  curl -fsS -m 10 -H 'Content-Type: application/json' \
       -d "{\"text\": \"${msg}\"}" "$WEBHOOK_URL" >/dev/null
  echo -e "$msg" | logger -t disk_alert
  exit 1
fi
echo "All filesystems below ${THRESHOLD}%"
```

```
# cron: every 10 minutes
*/10 * * * * WEBHOOK_URL=https://example.webhook/abc /opt/scripts/disk_alert.sh 80
```

Watch out: Also check inodes (`df -Pi`), because a disk can be "full" with free space left. Without some state, the script re-alerts every run, so keep a small marker file for cooldown. In real production this belongs in Prometheus/Zabbix (node_exporter `node_filesystem_avail_bytes`), and the script is only a fallback.

> ***What to say to the interviewer:*** *I parse `df -P`, skip tmpfs and overlay, strip the percent sign, and compare each filesystem against a threshold. If any cross it, I post one message to a Teams or Slack webhook and exit non-zero. I run it from cron every 10 minutes and I'd also check inode usage. In a mature setup this is a Prometheus or Zabbix alert rule, and the script is only a backup.*

Q17. Write a script to monitor a web service and alert on failure.

> **In simple words:** A robot that knocks on the website's door every minute. If nobody answers a few times in a row, it calls the on-call person.

Detailed answer

- Use `curl` with a hard timeout, and capture the HTTP status code and the response time.
- Retry a few times before alerting, so one network blip doesn't page anyone.

- Keep a state file so you alert once when it goes down and once when it recovers, not every minute.
- Related: restarting a service automatically when it is down is covered in the Linux chapter.

Example:

```
#!/usr/bin/env bash
# web_check.sh - check a URL, alert on failure and on recovery
set -uo pipefail

URL="${1:?usage: $0 <url> [expected_code]}"
EXPECTED="${2:-200}"
WEBHOOK_URL="${WEBHOOK_URL:?set WEBHOOK_URL}"
STATE="/var/tmp/web_check_$(echo "$URL" | md5sum | cut -c1-8).state"
RETRIES=3

notify() {
  curl -fsS -m 10 -H 'Content-Type: application/json' \
       -d "{\"text\": \"$1\"}" "$WEBHOOK_URL" >/dev/null || true
}

ok=false
for i in $(seq 1 "$RETRIES"); do
  read -r code ttime < <(curl -s -o /dev/null -m 10 \
      -w '%{http_code} %{time_total}' "$URL" || echo "000 0")
  if [[ "$code" == "$EXPECTED" ]]; then ok=true; break; fi
  sleep 5
done

if $ok; then
  [[ -f "$STATE" ]] && { notify "RECOVERED: $URL is back (HTTP $code, ${ttime}s)"; rm -f "$STATE"; }
  exit 0
fi

if [[ ! -f "$STATE" ]]; then
  notify "DOWN: $URL returned HTTP $code after $RETRIES attempts"
  date > "$STATE"
fi
exit 1
```

Watch out: A 200 status can still hide a broken page, so optionally grep the body for a keyword or check a `/health` endpoint that tests dependencies. Run the check from outside the server's own network too, or you'll miss DNS, TLS and load balancer problems. `curl` code `000` means connection or DNS failure, not an HTTP error.

> ***What to say to the interviewer:*** *I use curl with a 10-second timeout to capture the status code and latency, retry three times to avoid false alarms, and keep a state file so the team gets one DOWN message and one RECOVERED message. I also run the same check from outside our network, because that is what users experience. Blackbox exporter or synthetic monitoring is the production-grade version.*

Q18. Write a script to rotate logs and keep only the last 5 files.

> **In simple words:** Like a stack of 5 notebooks. When the current one is full you put it on top of the stack, the oldest one at the bottom gets thrown away, and you start a fresh notebook.

Detailed answer

- Shift `app.log.4 -> app.log.5` ... `app.log.1 -> app.log.2`, then `app.log -> app.log.1`, and delete anything beyond 5.
- Use copy-then-truncate if the app keeps the file open. Otherwise it keeps writing to the renamed file.
- Compress old copies to save space.
- In real life, `logrotate` does exactly this, so show both.

Example:

```
#!/usr/bin/env bash
# rotate_log.sh - keep the last N rotations of a log file
set -euo pipefail

LOG="${1:?usage: $0 <logfile> [keep]}"
KEEP="${2:-5}"
[[ -f "$LOG" ]] || { echo "no such file: $LOG" >&2; exit 1; }

rm -f "${LOG}.${KEEP}.gz"                       # drop the oldest
for ((i=KEEP-1; i>=1; i--)); do
  [[ -f "${LOG}.${i}.gz" ]] && mv "${LOG}.${i}.gz" "${LOG}.$((i+1)).gz"
done
cp -p "$LOG" "${LOG}.1" && : > "$LOG"           # copytruncate: app keeps its fd
gzip -f "${LOG}.1"
echo "rotated $LOG, keeping $KEEP files"
```

```
# The logrotate equivalent: /etc/logrotate.d/myapp
/var/log/myapp/app.log {
    daily
    rotate 5
    compress
    missingok
    notifempty
    copytruncate
}
```

Watch out: With `copytruncate`, lines written between the copy and the truncate can be lost, so a signal-based reopen (`postrotate` + `kill -HUP` or `systemctl reload`) is cleaner if the app supports it. Test with `logrotate -d` for a dry run. In containers, don't rotate inside the container; log to stdout and let the runtime rotate.

> ***What to say to the interviewer:*** *I shift the numbered files up by one, delete the one beyond the keep count, copy the live log to* `.1`*, truncate the original so the app keeps writing, and gzip the copy. In production I'd use a logrotate config with* `rotate 5`*,* `compress` *and* `copytruncate`*, or better, a postrotate reload so no lines are lost.*

Q19. Write a script to fetch the top CPU-consuming processes and email them.

> **In simple words:** A snapshot of the "who is eating the CPU" leaderboard, sent to your inbox, so when someone asks "why was the server slow at 3 AM?" you have evidence.

Detailed answer

- `ps -eo pid,ppid,user,%cpu,%mem,etime,cmd --sort=-%cpu | head -n 11` gives the header plus the top 10.

- Add a load average and uptime line for context.
- Send it with `mail` (mailx) or `sendmail`, or post it to Teams/Slack as an alternative.
- Trigger it from cron, or better, only when load goes above a threshold.

Example:

```
#!/usr/bin/env bash
# top_cpu_report.sh - email the top CPU consumers, optionally only above a load threshold
set -euo pipefail

TO="${1:?usage: $0 <email> [load_threshold]}"
LOAD_LIMIT="${2:-0}"
HOST="$(hostname -f)"
load1="$(cut -d' ' -f1 /proc/loadavg)"

# only report when 1-min load is above the limit (0 = always)
if awk -v l="$load1" -v t="$LOAD_LIMIT" 'BEGIN{exit !(l < t)}'; then
  exit 0
fi

report="$(
  echo "Host: $HOST   Time: $(date '+%F %T')   Load: $(cut -d' ' -f1-3 /proc/loadavg)"
  echo "CPUs: $(nproc)"
  echo
  ps -eo pid,ppid,user,%cpu,%mem,etime,cmd --sort=-%cpu | head -n 11 | cut -c1-150
)"

echo "$report" | mail -s "[${HOST}] Top CPU processes (load ${load1})" "$TO"
echo "report sent to $TO"
```

Watch out: `%cpu` in `ps` is the average over the process lifetime, not the current usage. For a "right now" view, use `top -b -n 2 -d 1` and take the second iteration. `mail` needs a working MTA or relay (postfix/SES SMTP). Compare the load against `nproc`: a load of 8 is fine on 16 cores and bad on 2.

> ***What to say to the interviewer:*** *I use* `ps` *sorted by CPU and take the top 10 with pid, user and command, add the load average and core count for context, and email it with mailx. I trigger it only when the load crosses a threshold, and I mention that* `ps %cpu` *is a lifetime average, so for a live picture I use* `top -b` *in batch mode.*

Q20. Write a script to tail multiple log files in parallel.

> **In simple words:** Watching several CCTV screens on one monitor, with each line labelled by which camera it came from.

Detailed answer

- `tail -F file1 file2` already follows several files and prints `==> file <==` headers. `-F` also survives log rotation.
- For a cleaner view, run one `tail` per file in the background and prefix each line with the file name.
- `trap` kills all the background tails when you press Ctrl+C.
- `multitail` or `lnav` are nicer interactive tools, and `journalctl -f -u a -u b` does the same for systemd services.

Example:

```
#!/usr/bin/env bash
# multitail.sh - follow several logs at once with a per-file prefix
set -euo pipefail
(( $# >= 1 )) || { echo "usage: $0 <log1> [log2 ...]" >&2; exit 2; }

pids=()
trap 'kill "${pids[@]}" 2>/dev/null; exit 0' INT TERM

for f in "$@"; do
  name="$(basename "$f")"
  tail -n 0 -F "$f" 2>/dev/null | sed -u "s/^/[${name}] /" &
  pids+=("$!")
done
wait
```

```
./multitail.sh /var/log/nginx/access.log /var/log/nginx/error.log /var/log/app/app.log | grep -i err
    or
```

Watch out: Use `sed -u` (unbuffered) or `stdbuf -oL`, otherwise output comes out in delayed chunks. `-F` vs `-f`: `-f` stops following after rotation. A file name containing `/` or `&` would break the sed expression, so I use the basename. For many servers, use a central log system (Loki/ELK) instead of tailing by hand.

> ***What to say to the interviewer:*** *For a quick look,* `tail -F` *with several files is enough. For a readable stream, I start one* `tail -F` *per file in the background, prefix each line with the file name using unbuffered sed, and trap Ctrl+C to kill all of them. Across many servers I'd use Loki or journalctl rather than tailing each one.*

Q21. Write a script to find zombie processes and clean them up.

> **In simple words:** A zombie is a finished worker whose manager never signed the exit paperwork. You can't fire someone who has already left; you have to nudge the manager (the parent) to do the paperwork, or replace the manager.

Detailed answer

- Zombies have state `Z` in `ps`. They use no CPU or memory, only a PID slot.
- **You cannot kill a zombie**; it is already dead. `kill -9` on it does nothing.
- Fix: send `SIGCHLD` to the parent so it calls `wait()`. If the parent is buggy, restart or kill the parent; the zombies are then re-parented to init/systemd, which reaps them.
- See the Linux chapter's zombie and parent-child process questions for the theory.

Example:

```
#!/usr/bin/env bash
# zombies.sh - list zombies and their parents; optionally nudge or kill the parents
set -euo pipefail
ACTION="${1:-report}"          # report | sigchld | kill-parent

mapfile -t rows < <(ps -eo pid=,ppid=,stat=,comm= | awk '$3 ~ /^Z/')
if (( ${#rows[@]} == 0 )); then echo "no zombie processes"; exit 0; fi

printf '%-8s %-8s %-6s %s\n' PID PPID STAT CMD
printf '%s\n' "${rows[@]}"

mapfile -t parents < <(printf '%s\n' "${rows[@]}" | awk '{print $2}' | sort -u)
for p in "${parents[@]}"; do
  echo "parent $p: $(ps -o comm= -p "$p") has $(printf '%s\n' "${rows[@]}" \
       | awk -v p="$p" '$2==p' | wc -l) zombie(s)"
  case "$ACTION" in
    sigchld)     kill -s SIGCHLD "$p" && echo "  sent SIGCHLD to $p" ;;
    kill-parent) [[ "$p" -ne 1 ]] && kill -TERM "$p" && echo "  terminated parent $p" ;;
  esac
done
```

Watch out: Never kill PID 1. Killing the parent kills the real service, so do it in a maintenance window or restart via systemd. A few short-lived zombies are normal. Thousands of them mean a parent is leaking and can exhaust the PID limit (`kernel.pid_max`), so fix the code or run with an init like `tini` in containers.

> ***What to say to the interviewer:*** *I list processes in state Z with their parent PIDs. Since a zombie is already dead, I first send SIGCHLD to the parent so it reaps its children. If that doesn't work, I restart or kill the parent and let init reap them. The real fix is in the parent's code, or using tini as PID 1 in containers.*

Q22. Write a script to monitor network bandwidth in real time.

> **In simple words:** Reading the electricity meter twice, one second apart. The difference tells you how fast the power is being used right now.

Detailed answer

- The kernel keeps byte counters per interface in `/sys/class/net/<if>/statistics/rx_bytes` and `tx_bytes`.
- Read them, sleep 1 second, read them again; the difference is bytes per second. Multiply by 8 for bits.
- No packages are needed, so it works on minimal servers and inside containers.
- Tools like `iftop` (per connection), `nload`, `bmon`, `sar -n DEV 1` and `vnstat` (history) do this with more detail.

Example:

```
#!/usr/bin/env bash
# bw.sh - live RX/TX rate for an interface, refreshed every second
set -euo pipefail

IFACE="${1:-$(ip route show default | awk '{print $5; exit}')}"
INTERVAL="${2:-1}"
S="/sys/class/net/${IFACE}/statistics"
[[ -d "$S" ]] || { echo "interface not found: $IFACE" >&2; exit 1; }

human() { numfmt --to=iec --suffix=B/s "$1"; }

rx1=$(<"$S/rx_bytes"); tx1=$(<"$S/tx_bytes")
echo "Monitoring $IFACE every ${INTERVAL}s (Ctrl+C to stop)"
while sleep "$INTERVAL"; do
  rx2=$(<"$S/rx_bytes"); tx2=$(<"$S/tx_bytes")
  rx=$(( (rx2 - rx1) / INTERVAL )); tx=$(( (tx2 - tx1) / INTERVAL ))
  printf '%s  RX %-12s TX %-12s (%d / %d Mbit/s)\n' "$(date +%T)" \
    "$(human "$rx")" "$(human "$tx")" $(( rx*8/1000000 )) $(( tx*8/1000000 ))
  rx1=$rx2; tx1=$tx2
done
```

```
Monitoring eth0 every 1s (Ctrl+C to stop)
10:42:01  RX 1.2MB/s      TX 350KB/s      (10 / 2 Mbit/s)
```

Watch out: Know the instance's bandwidth limit (on AWS it depends on instance size; watch `bw_in_allowance_exceeded` via `ethtool -S`). Bytes vs bits: network links are sold in bits. Counters are per interface, so bonded or VLAN interfaces and the docker0 bridge each have their own.

> ***What to say to the interviewer:*** *I read the rx and tx byte counters from sysfs, sleep a second, read them again, and print the difference as bytes and megabits per second. It needs no extra packages. For per-connection detail I use iftop, and for history sar or vnstat. In production, node_exporter exposes the same counters to Prometheus.*

Q23. Write a script that alerts if a Docker container is not running.

> **In simple words:** A headcount at the start of every shift. If a named container is missing or unhealthy, the supervisor gets a message.

Detailed answer

- `docker inspect -f '{{.State.Status}}'` returns `running`, `exited`, `restarting` and so on. It errors if the container doesn't exist.
- Also read the health status (`{{.State.Health.Status}}`) when the image has a HEALTHCHECK, because "running" is not the same as "healthy".
- Alert with the exit code, restart count and last log lines to speed up triage.
- Optionally try `docker start` once, but still alert so a human looks at the root cause.

Example:

```
#!/usr/bin/env bash
# container_check.sh - alert when containers are missing, stopped or unhealthy
set -uo pipefail

WEBHOOK_URL="${WEBHOOK_URL:?set WEBHOOK_URL}"
(( $# >= 1 )) || { echo "usage: $0 <container> [container ...]" >&2; exit 2; }
problems=()

for c in "$@"; do
  if ! status=$(docker inspect -f '{{.State.Status}}' "$c" 2>/dev/null); then
    problems+=("$c: NOT FOUND"); continue
  fi
  health=$(docker inspect -f '{{if .State.Health}}{{.State.Health.Status}}{{end}}' "$c")
  if [[ "$status" != "running" || "$health" == "unhealthy" ]]; then
    code=$(docker inspect -f '{{.State.ExitCode}}' "$c")
    restarts=$(docker inspect -f '{{.RestartCount}}' "$c")
    last=$(docker logs --tail 3 "$c" 2>&1 | tr '\n"' ' _' | cut -c1-200)
    problems+=("$c: status=$status health=${health:-n/a} exit=$code restarts=$restarts | $last")
  fi
done

if (( ${#problems[@]} )); then
  msg="Container alert on $(hostname): $(printf '%s; ' "${problems[@]}")"
  curl -fsS -m 10 -H 'Content-Type: application/json' -d "{\"text\": \"$msg\"}" \
       "$WEBHOOK_URL" >/dev/null
  echo "$msg"; exit 1
fi
echo "all containers running"
```

Watch out: The user running it needs Docker socket access, and membership of the `docker` group is effectively root, so run it as a dedicated service user. A container in a restart loop shows `restarting` or keeps a rising RestartCount, so alert on that too. For many hosts use cAdvisor + Prometheus (`time() - container_last_seen`) instead of scripts.

> ***What to say to the interviewer:*** *For each expected container I use docker inspect to read the status and health, and if it's missing, stopped or unhealthy I send one webhook message with the exit code, restart count and last log lines. A restart policy handles recovery, and the script makes sure a human knows. At scale I'd rely on cAdvisor metrics and Prometheus alerts.*

Q24. Write a script to auto-deploy a static website using rsync.

> **In simple words:** Instead of repainting the shop while customers are inside, you set up the new display in a back room and then swap the sign on the door in one second.

Detailed answer

- Build locally (or in CI), then rsync to a new timestamped release folder on the server.
- Switch a `current` symlink atomically to the new release, so users never see half-uploaded files.
- Keep the last few releases for instant rollback.
- Use `--dry-run` first, and use `--delete` so removed files disappear from the release.

Example:

```
#!/usr/bin/env bash
# deploy_static.sh - atomic static site deploy with rsync + symlink switch
set -euo pipefail

SRC="${1:?usage: $0 <build_dir> <user@host> [base_dir]}"
TARGET="${2:?missing user@host}"
BASE="${3:-/var/www/site}"
KEEP=5
REL="$(date +%Y%m%d%H%M%S)"
SSH="ssh -o BatchMode=yes -o ConnectTimeout=10"

[[ -f "$SRC/index.html" ]] || { echo "build dir looks wrong: $SRC" >&2; exit 1; }

echo "Dry run:"; rsync -azn --delete -e "$SSH" "$SRC"/ "$TARGET:$BASE/releases/$REL/" | tail -5

$SSH "$TARGET" "mkdir -p '$BASE/releases/$REL'"
# --link-dest reuses unchanged files from the current release (fast, saves space)
rsync -az --delete --link-dest="$BASE/current/" -e "$SSH" \
      "$SRC"/ "$TARGET:$BASE/releases/$REL/"

$SSH "$TARGET" "ln -sfn '$BASE/releases/$REL' '$BASE/current.tmp' \
  && mv -T '$BASE/current.tmp' '$BASE/current' \
  && ls -1dt '$BASE'/releases/* | tail -n +$((KEEP+1)) | xargs -r rm -rf"

echo "Deployed release $REL"
# Rollback: ln -sfn $BASE/releases/<previous> $BASE/current.tmp && mv -T ... current
```

Watch out: `mv -T` of a symlink is atomic; `ln -sfn` alone on the live link has a tiny window. Point the nginx root at `$BASE/current`. Use a deploy-only SSH key restricted in `authorized_keys`. A trailing slash on the rsync source means "copy the contents", not the folder itself. With a CDN in front, invalidate the cache after the switch.

> ***What to say to the interviewer:*** *I rsync the build output into a new timestamped release directory over SSH, using --link-dest so unchanged files are hard-linked, then switch a current symlink atomically with mv -T, so users never see a half-deployed site. I keep the last five releases, so rollback is just pointing the symlink back, and I run a dry run first.*